

推导完全分类

你的诊断抓住了核心：Q4 解决的是元素 ID 的确定性，却没有解决跨 bar 的身份连续性。
严格说，这不是 AncOK 小 bug，而是状态转移层缺少元素延拓映射。

你的总结是对的，而且可以把这个问题表述成一个很干净的形式化冲突：

$$\boxed{\text{确定性 ID} \neq \text{跨 bar 持久身份}}$$

Q4 修了：

$$\text{Id}(e)$$

的确定性，但没有保证：

$$e_i \in E_i \implies \exists e_{i+1} \in E_{i+1} \text{ with same identity.}$$

也就是说，身份证号生成规则稳定了，但“这个人明天还在组织里”没有保证。

1. 形式化地说，当前问题在哪里

令第 i 根 bar 后的全量重建元素集为：

$$E_i = \text{extract_elements}(D_i).$$

策略持仓腿引用的是某个元素 ID：

$$L_i = (\text{id}(e), \text{side}, \text{qty}, \text{parent}, \dots).$$

下一根 bar 来了以后，系统重新生成：

$$E_{i+1} = \text{extract_elements}(D_{i+1}).$$

若：

$$\text{id}(e) \notin \text{Id}(E_{i+1}),$$

则 held 腿被判定为：

$$\text{Stale.}$$

Q4 的新逻辑是诚实的：

$$\text{Stale} \Rightarrow \text{prune},$$

而不是旧逻辑那样伪造：

$$\text{parent} = \text{None.}$$

所以 Q4 修掉的是一个假根污染问题。

但剩下的问题是：

$$\boxed{E_i \text{ 与 } E_{i+1} \text{ 之间没有持久嵌入。}}$$

也就是缺少一个映射：

$$\phi_i : E_i^{\text{live}} \rightarrow E_{i+1}^{\text{live}}$$

满足：

$$\text{id}(\phi_i(e)) = \text{id}(e).$$

你给的数据：

$$\text{held_exact}_{\text{CL}} = 0.8\%, \quad \text{held_exact}_{\text{BTC}} = 27.8\%$$

说明绝大多数 held legs 在下次全量树里已经找不到同一元素。

因此：

$\text{depth} > 0$ 腿不是被交易逻辑杀死的，而是被身份层杀死的。

2. 这不是 AncOK 的 bug

当前 Q4 后的 AncOK 行为其实是正确的。

如果一个 held 子腿的父不存在，且你又拒绝伪造：

$$\text{parent} = \text{None},$$

那么 AncOK 必须剪掉它。

因为祖先闭合要求：

$$v \in A_t \implies \text{Anc}(v) \subseteq A_t.$$

如果父不在：

$$p(v) \notin A_t,$$

那么：

$$v \notin \text{AncOK}(A_t).$$

所以这条链是严格成立的：

旧腿 ID 找不到 $\Rightarrow$ Stale $\Rightarrow$ 不入 raw pool $\Rightarrow$ 祖先缺失 $\Rightarrow$ $\text{depth} > 0$ 被 AncOK 剪掉 $\Rightarrow$ $\#5\alpha = 1$

这不是剪枝逻辑错，而是身份模型没有满足策略记忆所需的不变量。

3. 真正缺少的不变量

策略如果要跨 bar 持有 $\text{depth} > 0$ 的腿，必须有下面这个不变量：

Live-leg Persistence

即：

L_i live and references $e \implies \forall j > i, \text{ until explicit close, } e \in \mathcal{P}_j.$

这里的：

$$\mathcal{P}_j$$

不能是每 bar 重新 extract 出来的瞬时元素集 E_j ，而必须是一个跨 bar 持久元素层。

也就是：

$$E_j = \text{snapshot},$$

$$\mathcal{P}_j = \text{persistent registry}.$$

当前架构只有：

$$E_j.$$

缺少：

$$\mathcal{P}_j.$$

所以 held 腿没有稳定归宿。

4. 关键修复：把“结构快照”和“操作记忆”分离

应该引入两层：

Snapshot Layer

和：

Persistent Element Layer.

Snapshot Layer

每 bar 仍然可以全量重建：

$$E_i = \text{extract_elements}(D_i).$$

这是当前结构视图。

Persistent Element Layer

维护一个持久注册表：

$$\mathcal{P}_i.$$

它满足：

$$\mathcal{P}_{i+1} = \text{Reconcile}(\mathcal{P}_i, E_{i+1}).$$

held 腿引用的不是 E_i 中的临时对象，而是：

$$\text{pid} \in \mathcal{P}_i.$$

也就是说，订单、腿、父子关系、三阶段资本账本都挂在 persistent element 上，而不是每 bar 的 Vec index 或 snapshot-only ID 上。

5. ID 不能包含易变父路径

如果 ElementId 包含：

- 当前 Vec index；
- 当前 root path；
- 当前 parent path；
- 当前最高级别容器；
- 当前树形位置；

那么一旦更高级别涌现或树重构，旧元素的 ID 就会变。

严格说，元素身份应该尽量由内禀边界决定：

$\text{pid}(e) = H(\ell(e), \text{start_anchor}(e), \text{end_anchor}(e), \delta(e), \text{kind}(e), \text{tie_break}(e)).$

不要把 parent 放进 element identity。

父关系应该是另一张表：

$$\text{parent}_i : \mathcal{P}_i \rightarrow \mathcal{P}_i \cup \{\perp\}.$$

这样当更高级别涌现时，只更新关系：

$$\text{parent}_i(e) = p_1 \longrightarrow \text{parent}_{i+1}(e) = p_2,$$

而不是把 e 本身重命名。

一句话：

元素 ID 表示“它是谁”；parent 表示“它现在被谁包含”。

不能把这两件事混在一个 ID 里。

6. 更严格的操作语义：held 腿应挂在 operation parent 上

还有一层更重要。

之前我们已经讨论过：操作语义里的短差、顺势、反向，不应当绑定未来最终父结构，而应当绑定入场时的操作容器。

所以 held 腿应该保存：

$$\text{op_parent}(L)$$

而不是每 bar 重建后的结构 parent。

例如某个子腿在时刻 i 是基于容器 c_i 开出来的：

$$L_i = (e, \text{op_parent} = c_i, \text{role} = S, \dots).$$

之后更高级别涌现，最终结构 parent 变化：

$$p_{i+1}^{\text{str}}(e) \neq c_i.$$

这不应该自动让 held 腿 stale。

因为腿的合法性来自：

入场时的操作语义

而不是未来重构后的树路径。

所以应区分：

$$p^{\text{op}}(e)$$

和：

$$p_i^{\text{str}}(e).$$

- p^{op} ：开腿时使用的操作父容器，驱动持仓生命周期；
- p_i^{str} ：当前 snapshot 中的结构父关系，可随更高级别涌现而变化。

AncOK 对 held legs 应该检查：

$$p^{\text{op}}$$

是否仍在 persistent registry 中有效，而不是要求它一定出现在当前 snapshot E_i 里。

7. 修复后的关键不变量

持久元素层需要满足这些不变量。

11. 持久身份不变量

若腿 L 未关闭，且引用元素 e ，则：

$$\text{pid}(e) \in \mathcal{P}_i$$

对所有后续 bar 成立，直到该腿被显式关闭。

12. 元素方向不变

$$\delta_i(e) = \delta_j(e)$$

只要 e 是同一个 persistent element。

高级别涌现不能改写低级别元素方向。

13. parent 是关系，不是身份

$$\text{pid}(e) \text{ 不依赖 } \text{parent}(e).$$

重构只允许改变：

$$\text{parent}_i(e),$$

不能改变：

$$\text{pid}(e).$$

14. 操作父容器持久

若 held 子腿 L 有：

$$\text{op_parent}(L) = c,$$

则在腿未关闭期间：

$$c \in \mathcal{P}_i$$

或者 c 被一个显式 successor 替代：

$$c \rightsquigarrow c',$$

并且该替换由确定性迁移表记录。

15. AncOK 作用在 persistent active set 上

AncOK 不应只看当前 snapshot：

$$E_i.$$

而应看：

$$A_i^{\text{persistent}}.$$

也就是：

$$A_i^{\text{ok}} = \text{AncOK}_{\mathcal{P}_i}(A_i).$$

8. 修复后为什么 depth>0 不会被系统性剪掉

设一个 $\text{depth} > 0$ 腿 L 在 bar i 开启，引用元素 e ，操作父为 c 。

即：

$$L_i = (e, c, \text{depth} > 0, \dots).$$

假设它没有触发显式平仓条件。

由 I1:

$$e \in \mathcal{P}_j, \quad j > i.$$

由 I4:

$$c \in \mathcal{P}_j$$

或:

$$c \rightsquigarrow c_j$$

存在确定性 successor。

因此, held 腿不因 snapshot missing 被判 Stale。

即使:

$$e \notin E_j,$$

也只是:

$$\text{snapshot_present}(e) = 0,$$

而不是:

$$\text{persistent_alive}(e) = 0.$$

所以:

$$e \notin E_j \setminus \text{centernot} \Rightarrow \text{Stale}.$$

Stale 应该定义为:

$$\text{Stale}(L) \iff \text{pid}(e) \notin \mathcal{P}_j \text{ or explicit invalidation.}$$

不是:

$$\text{pid}(e) \notin E_j.$$

因此 $\text{depth} > 0$ 子腿不会因为全量树重构被系统性剪掉。

9. 最小工程修复与彻底修复

最小修复: persistent overlay

不用立刻重写 extract_elements。

保留现有:

$$E_i = \text{extract_elements}(D_i).$$

但增加:

$$\mathcal{P}_i$$

作为 overlay。

每 bar 做:

$$\mathcal{P}_{i+1} = \text{merge}(\mathcal{P}_i, E_{i+1}, \text{held legs}).$$

规则:

1. 当前 snapshot 中能匹配到的元素，刷新其 snapshot fields;
2. held 引用但 snapshot 找不到的元素，不删除，标记为：

Detached

或：

SnapshotMissing.

3. Detached held leg 不允许开新子腿，但允许按照自己的退出规则继续存在或被显式关闭；
4. AncOK 对 held legs 使用 p^{op} ，不使用当前 snapshot 的 parent；
5. 只有显式 close、risk close、successor invalidation 才删除 persistent element。

这能快速验证：

$$\text{Stale rate} \downarrow, \quad \text{depth} > 0 \text{ admission} \uparrow, \quad \#5\alpha \neq 0.$$

代价是：snapshot 和 persistent registry 会短期并存，有些 detached 元素不在当前结构视图里。

但这是诚实的：它表达的是“操作记忆还在，当前重建树没把它展示出来”。

彻底修复：增量 extract_elements

最终正确形态是：

$$\mathcal{P}_{i+1} = \text{IncExtract}(\mathcal{P}_i, Y_{i+1}).$$

也就是：

不每 bar 重建完整元素树，而是增量更新前沿。

已完成、已确认的历史元素保持不变：

$$e \in \mathcal{P}_i^{\text{confirmed}} \implies e \in \mathcal{P}_{i+1}^{\text{confirmed}}$$

且：

$$\text{pid}_{i+1}(e) = \text{pid}_i(e).$$

只允许活动 frontier 变化：

$$\mathcal{F}_i \rightarrow \mathcal{F}_{i+1}.$$

更高级别涌现时：

$$g_1, \dots, g_m \mapsto G,$$

只新增：

$$G$$

和包含边：

$$g_j \sqsubset G,$$

不删除：

$$g_j.$$

这是最符合我们前面“自底向上生成、更高级别涌现不改写低级别方向”的数学定义。

代价就是你说的：工程量大，而且 bit-exact 风险高，因为需要处理：

- 最高级变化；
- frontier 回滚；
- index remapping；
- 多义结构规范化；
- active/confirmed 分离；
- parent 迁移；
- historical snapshot 对齐。

10. Q4 后 spec 应该怎么改

当前 spec 隐含了：

held leg validity $\iff$ ID exists in current full snapshot.

这应该改成：

held leg validity $\iff$ ID exists in persistent registry and has not been explicitly invalidated.

形式上：

$$\text{ValidHeld}_i(L) := \text{pid}(L) \in \mathcal{P}_i \wedge \neg \text{Invalidated}_i(L).$$

而不是：

$$\text{pid}(L) \in E_i.$$

然后：

$$\text{SnapshotPresent}_i(L) := \text{pid}(L) \in E_i$$

只是一个附加状态。

于是 held leg 有四种状态：

状态	含义
LivePresent	持久存在，当前快照也存在
LiveDetached	持久存在，但当前快照找不到
Closed	显式平仓结束
Invalidated	结构或风险显式作废

Q4 目前把：

LiveDetached

误处理成：

Stale/Invalidated.

这就是 depth > 0 被杀光的直接原因。

11. 严格证明：persistent registry 修复 Stale 链

定义：

$$H_i = \text{bar } i \text{ 后 held legs.}$$

每个 held leg $L \in H_i$ 引用：

$$\text{pid}(L) \in \mathcal{P}_i.$$

假设：

$$L \in H_i$$

且没有触发显式关闭：

$$\neg \text{CloseSignal}_{i+1}(L),$$

$$\neg \text{RiskClose}_{i+1}(L).$$

persistent registry 规则保证：

$$\text{pid}(L) \in \mathcal{P}_{i+1}.$$

因此：

$$\text{ValidHeld}_{i+1}(L) = 1.$$

即使：

$$\text{pid}(L) \notin E_{i+1},$$

也只是：

$$\text{SnapshotPresent}_{i+1}(L) = 0,$$

不会推出：

$$\text{ValidHeld}_{i+1}(L) = 0.$$

所以：

$$L \not\Rightarrow \text{Stale}.$$

若 L 是 $\text{depth} > 0$ 子腿，则它的操作父：

$$\text{op_parent}(L)$$

也由 registry 保持或迁移。故：

$$\text{AncOK}_{\mathcal{P}_{i+1}}(L) = 1$$

只要其操作父没有显式关闭。

因此：

$$\text{full snapshot 重构} \not\Rightarrow \text{depth} > 0 \text{ 系统性剪枝}.$$

证毕。

12. 测试应怎样改

现在的 held_exact 指标其实很好，它已经暴露了问题。但它应该拆成三类：

held_snapshot_exact

held_registry_alive

held_operation_parent_alive

目标不是让：

$$\text{held_snapshot_exact} = 100\%.$$

如果仍然允许 full snapshot 重构，这个指标可能永远不高。

真正目标是：

held_registry_alive $\approx$ 100%

对于未显式关闭的腿。

以及：

held_operation_parent_alive $\approx$ 100%

对于未触发父级关闭的 $\text{depth} > 0$ 腿。

测试矩阵应该包括：

1. 更高级别涌现测试

低级别元素 ID 不变，只新增高一级父容器。

2. reparent 测试

parent edge 可变，但 child pid 不变。

3. held detached 测试

snapshot 找不到 held 元素，registry 仍保留，腿不被判 stale。

4. no fake root 测试

missing parent 不能伪造成 parent:None。

5. op_parent AncOK 测试

held 子腿按入场操作父容器通过 AncOK，而不是按当前 snapshot parent。

6. explicit close 测试

只有 close signal / risk close / invalidation 才能让 held leg 退出 registry live set。

13. 最终判断

你这段结论可以更精确地写成：

Q4 修复了 ID determinism 和 no-fake-root，但没有提供 live identity persistence。

当前失败不是因为：

Id

不确定，而是因为：

$$E_i \rightarrow E_{i+1}$$

缺少持久嵌入。

所以根因是：

per-bar full snapshot classification 与 held-leg operational memory 之间缺少 persistent reg

修复方向也正如你说的：

跨 bar 持久元素层

或者更彻底地：

增量 extract_elements, confirmed prefix immutable, frontier only mutable。

在没有这个层之前，Q4 的“诚实 prune”会继续把 $\text{depth} > 0$ 腿剪光，因为它只能看到：

当前重建树里没有这个人。

而严格操作语义真正需要的是：

这个操作腿引用的 persistent element 是否仍然 live。

这两个问题不能混在一起。



给我一个严格的推导和数学证明。

我会先形式化快照层、持久层、持仓腿、AncOK 与 Stale 判定，再证明 Q4 必然剪掉 $\text{depth} > 0$ 。随后证明伪造 $\text{parent} = \text{None}$ 不成立，并给出持久元素层的充要修复条件与全定义策略证明。

下面给出一个严格版本。核心结论是：

Q4 修复了 ID 确定性和不伪造根，但没有修复“跨 bar 持久身份”。

因此在当前架构下， $\text{depth} > 0$ 腿被剪光不是偶然，而是由 snapshot-only 身份判定与 AncOK 共同推出的必然结果。

1. 基本对象

用 bar 下标 $i \in \mathbb{N}$ 。

每个 bar 后，塔结构为

$$D_i.$$

全量重建元素树为

$$E_i = \text{Extract}(D_i).$$

每个元素有确定性 ID：

$$\text{id}_i : E_i \rightarrow \text{Id}.$$

Q4 修复的第一个点可以形式化为：

$$D_i = D'_i \implies \text{Extract}(D_i) = \text{Extract}(D'_i) \text{ 且 ID 相同。}$$

也就是：

$$\text{同一个输入快照，产出同一个元素集合和同一个 ID。}$$

但这只说明快照内确定性，不说明跨 bar 持久性。

跨 bar 持久性要求的是更强条件：

$$e \in E_i \text{ 且仍应存活} \implies \exists e' \in E_{i+1} : \text{id}_{i+1}(e') = \text{id}_i(e).$$

记这个条件为：

$$\text{Persist}(i, i + 1).$$

Q4 没有保证它。

2. held 腿与 snapshot-only stale 判定

令 H_i 为 bar i 后的 held legs 集合。

每条腿 $L \in H_i$ 引用一个元素 ID：

$$\epsilon(L) \in \text{Id}.$$

若 L 是 $\text{depth} > 0$ 的子腿，它还有操作父 ID：

$$\pi(L) \in \text{Id}.$$

当前 Q4 后的 snapshot-only 存活判定本质是：

$$\text{Present}_{i+1}(L) := \epsilon(L) \in \text{Id}(E_{i+1}).$$

其中：

$$\text{Id}(E_{i+1}) = \{\text{id}_{i+1}(e) : e \in E_{i+1}\}.$$

如果：

$$\epsilon(L) \notin \text{Id}(E_{i+1}),$$

则：

L 被判为 **Stale**.

Q4 的第二个修复是：

$$\text{Stale} \Rightarrow \text{prune},$$

而不是旧逻辑：

$$\text{Stale} \Rightarrow \text{parent} := \text{None}.$$

这一步是正确的，因为伪造 $\text{parent}=\text{None}$ 会把一个本来是子腿的东西伪装成根腿，破坏祖先闭合和角色语义。

3. AncOK 的严格定义

令 A 是某个候选活动腿集合。

每个 $\text{depth} > 0$ 腿 L 有父腿或父元素：

$$\pi(L).$$

祖先链记为：

$$\text{Anc}(L) = \{\pi(L), \pi^2(L), \dots\}$$

直到最高可见父级。

定义：

$$\text{AncOK}(A) = \{L \in A : \text{Anc}(L) \subseteq A\}.$$

也就是：

$$L \in \text{AncOK}(A) \iff L \in A \text{ 且所有祖先都在 } A.$$

所以如果父缺失：

$$\pi(L) \notin A,$$

则：

$$L \notin \text{AncOK}(A).$$

4. Q4 后 $\text{depth} > 0$ 腿被剪的定理

定理 1： snapshot-only 架构下， $\text{depth} > 0$ 腿存活当且仅当整条祖先链在新快照中存在

设 Q4 规则为：

1. held 腿如果在 E_{i+1} 中找不到自身 ID，则被判 Stale 并 prune；
2. 不允许把 missing parent 伪造成 parent=None；
3. 最终活动集合经过 AncOK。

则对任意 held 腿 $L \in H_i$ ，若它在 $i+1$ 后仍要保留，必须满足：

$$\{\epsilon(L)\} \cup \text{AncId}(L) \subseteq \text{Id}(E_{i+1}).$$

其中：

$$\text{AncId}(L) = \{\epsilon(P) : P \in \text{Anc}(L)\}.$$

证明

若：

$$\epsilon(L) \notin \text{Id}(E_{i+1}),$$

则由 Q4 stale 规则：

L 被 prune.

所以 L 不可能存活。

若 L 自己存在，但某个祖先 $P \in \text{Anc}(L)$ 不存在，即：

$$\epsilon(P) \notin \text{Id}(E_{i+1}),$$

则 P 被判 stale 并 prune，因此：

$$P \notin A_{i+1}^{\text{raw}}.$$

而 L 要通过 AncOK 必须满足：

$$\text{Anc}(L) \subseteq A_{i+1}^{\text{raw}}.$$

但 $P \notin A_{i+1}^{\text{raw}}$ ，矛盾。

所以 L 不能通过 AncOK。

因此，若 L 存活，则 L 自己以及它所有祖先的 ID 都必须存在于新快照中。

证毕。

推论 1：只要 held_exact 很低，depth > 0 准入必然很低

令：

$$r_i = \frac{|\{\epsilon(L) : L \in H_i\} \cap \text{Id}(E_{i+1})|}{|\{\epsilon(L) : L \in H_i\}|}.$$

这是 held_exact。

对 depth d 的腿，存活至少需要：

$$d + 1$$

个相关 ID 同时存在：自己、父、父的父、直到根。

因此存活集合满足：

$$\text{Survive}_{d>0} \subseteq \{L : \epsilon(L) \in E_{i+1}\} \cap \{L : \epsilon(\pi(L)) \in E_{i+1}\} \cap \dots$$

所以 $\text{depth} > 0$ 的存活率上界受 held_exact 严重限制。

在你的数据里：

$$\text{CL held_exact} = 0.8\%,$$

这意味着 snapshot-only 模型中，绝大多数 held 腿连自身都找不到，更不用说祖先链同时存在。

于是：

$$\text{depth} > 0 \text{ 腿被系统性剪掉。}$$

如果策略 #5 alpha 只来自 $\text{depth} > 0$ 反向对冲腿，则：

$$\text{depth} > 0 \text{ 准入} = 0 \implies \#5\alpha = 0 \implies \Delta\text{Sharpe} = 0.$$

这不是交易逻辑没有 alpha，而是身份层把所有 $\text{depth} > 0$ 操作腿剪掉了。

5. Q4 为什么不是错修

Q4 的 prune 比伪造 $\text{parent}=\text{None}$ 更正确。

旧逻辑：

$$\pi(L) \text{ missing} \implies \pi(L) := \perp$$

会把一个 $\text{depth} > 0$ 子腿伪装成根腿。

但短差语义要求：

$$\text{短差} = \text{父仓保持} + \text{子级反向双开}.$$

如果父被改成 None，那么这个腿不再是短差腿，而被伪造成 ambient/root 腿。

形式上，原本应有：

$$\text{Role}(L) = S \iff \delta(L) = -\delta(\pi(L)).$$

伪造后：

$$\pi(L) = \perp,$$

则它会被分类为：

$$A.$$

因此：

$$S \neq A.$$

所以：

$$\text{伪造 parent=None 会破坏角色分类。}$$

Q4 诚实 prune 修掉的是这个 bug。

但是它暴露了另一个事实：

$$\text{当前 snapshot 层无法承载跨 bar 操作记忆。}$$

6. 确定性 ID 不推出跨 bar 稳定性的反例

这是关键逻辑点。

令 extractor 是确定性函数：

$$E_i = F(D_i).$$

确定性只表示：

$$D_i = D'_i \Rightarrow F(D_i) = F(D'_i).$$

但跨 bar 时：

$$D_i \neq D_{i+1}.$$

因此完全可能：

$$F(D_i) = \{a\},$$

$$F(D_{i+1}) = \{b\},$$

其中：

$$a \neq b.$$

这个 F 仍然是完全确定性的。

所以：

$$\text{deterministic ID} \setminus \text{centernot} \Rightarrow \text{persistent identity}.$$

Q4 给了元素“身份证号”，但没有保证这个元素下一根 bar 仍在新组织架构里。

7. 正确修复：引入持久元素层

必须把两层分开：

$$\text{snapshot element set } E_i$$

和：

$$\text{persistent registry } \mathcal{P}_i.$$

其中：

$$E_i = \text{Extract}(D_i)$$

仍然可以每 bar 全量重建。

但 held legs 引用的必须是：

$$\text{pid} \in \mathcal{P}_i,$$

而不是直接引用当前 snapshot E_i 。

持久注册表递归更新：

$$\mathcal{P}_{i+1} = \text{Reconcile}(\mathcal{P}_i, E_{i+1}, H_i).$$

8. 持久层必须满足的不变量

I1. 持久身份不变量

若腿 $L \in H_i$ 未显式关闭，且引用元素：

$$\epsilon(L) = u,$$

则：

$$u \in \mathcal{P}_{i+1}.$$

除非出现显式无效化：

$\text{CloseSignal}_{i+1}(L)$ 或 $\text{RiskClose}_{i+1}(L)$ 或 $\text{StructuralInvalidation}_{i+1}(L)$.

也就是：

$$\text{snapshot missing} \neq \text{persistent dead}.$$

12. snapshot 显示与持久存活分离

定义：

$$\text{SnapshotPresent}_i(u) := u \in \text{Id}(E_i).$$

定义：

$$\text{RegistryLive}_i(u) := u \in \mathcal{P}_i \wedge \text{status}_i(u) \in \{\text{LivePresent}, \text{LiveDetached}\}.$$

必须允许：

$$\text{RegistryLive}_i(u) = 1 \quad \text{且} \quad \text{SnapshotPresent}_i(u) = 0.$$

这就是 detached live element。

13. parent 是关系，不是身份

元素身份不应依赖当前 parent path。

元素 ID 应该由内禀边界决定，例如：

$$\text{pid}(e) = H(\ell(e), t_{\text{start}}(e), t_{\text{end}}(e), \delta(e), \text{kind}(e), \text{tie_break}(e)).$$

而 parent 应该是边表：

$$\text{par}_i : \mathcal{P}_i \rightarrow \mathcal{P}_i \cup \{\perp\}.$$

当更高级别涌现时，可以更新：

$$\text{par}_i(e) = p \quad \longrightarrow \quad \text{par}_{i+1}(e) = p',$$

但不能改写：

$$\text{pid}(e).$$

14. 操作父容器持久

held 短差腿 L 必须保存入场时的操作父：

$$\text{opParent}(L) \in \mathcal{P}_i.$$

它表示：

$$\text{这个腿当时是相对哪个操作容器开的。}$$

它不应被每 bar snapshot 重构后的结构 parent 替代。

要求：

$$L \text{ 未关闭} \implies \text{opParent}(L) \in \mathcal{P}_i$$

或者存在确定性 successor：

$$\text{opParent}(L) \rightsquigarrow \text{opParent}'(L).$$

15. AncOK 作用于 persistent active set

修复后 AncOK 应该定义为：

$$\text{AncOK}_{\mathcal{P}_i}(A) = \{L \in A : \text{OpAnc}(L) \subseteq A \cup \mathcal{P}_i^{\text{live}}\}.$$

至少对 held legs 来说，检查的是：

$$\text{opParent}(L)$$

是否 live，而不是它是否存在于当前 snapshot E_i 。

9. 修复后的状态分类

每个 held 引用元素 u 的状态定义为四类：

$$\Omega_i(u) \in \{\text{LivePresent}, \text{LiveDetached}, \text{Closed}, \text{Invalidated}\}.$$

严格定义：

$$\Omega_i(u) = \begin{cases} \text{Closed}, & \text{Closed}_i(u) = 1, \\ \text{Invalidated}, & \text{Closed}_i(u) = 0 \wedge \text{Invalidated}_i(u) = 1, \\ \text{LivePresent}, & \text{Closed}_i(u) = 0 \wedge \text{Invalidated}_i(u) = 0 \wedge u \in \text{Id}(E_i), \\ \text{LiveDetached}, & \text{Closed}_i(u) = 0 \wedge \text{Invalidated}_i(u) = 0 \wedge u \notin \text{Id}(E_i). \end{cases}$$

这四类互斥且穷尽：

$$\sum_{\omega \in \Omega} \mathbf{1}[\Omega_i(u) = \omega] = 1.$$

证明与优先级分类相同：按 Closed、Invalidated、Present、Detached 依次判定，恰好一个成立。

Q4 当前把：

LiveDetached

错误合并进：

Invalidated/Stale.

这就是系统性剪枝的数学根源。

10. 修复定理：持久层消除 snapshot 重构导致的系统性 stale

定理 2

假设持久层满足 I1–I5。

令 $L \in H_i$ 是一条 $\text{depth} > 0$ held 腿。若：

$$\begin{aligned} &\neg \text{CloseSignal}_{i+1}(L), \\ &\neg \text{RiskClose}_{i+1}(L), \\ &\neg \text{StructuralInvalidation}_{i+1}(L), \end{aligned}$$

并且其操作祖先链 live：

$$\text{OpAnc}(L) \subseteq \mathcal{P}_{i+1}^{\text{live}},$$

则 L 不会因为

$$\epsilon(L) \notin \text{Id}(E_{i+1})$$

而被 prune。

证明

由 I1, L 未显式关闭且未无效化，因此：

$$\epsilon(L) \in \mathcal{P}_{i+1}.$$

如果：

$$\epsilon(L) \in \text{Id}(E_{i+1}),$$

则 L 为：

$$\text{LivePresent}.$$

如果：

$$\epsilon(L) \notin \text{Id}(E_{i+1}),$$

则由 I2, 不能推出 L dead, 只能得到：

$$L \text{ 为 LiveDetached}.$$

因此：

$$\text{RegistryLive}_{i+1}(\epsilon(L)) = 1.$$

又由假设：

$$\text{OpAnc}(L) \subseteq \mathcal{P}_{i+1}^{\text{live}}.$$

根据 I5, AncOK 对 persistent live set 检查祖先链，所以：

$$L \in \text{AncOK}_{\mathcal{P}_{i+1}}(A).$$

因此 L 不会因为当前 snapshot 找不到自身或其操作父而被剪掉。

证毕。

11. $\text{depth} > 0$ 存活的归纳证明

定义 depth ：

$$d(L) = 0$$

表示无操作父的 ambient 腿；

$$d(L) > 0$$

表示子腿。

命题

若每条未关闭腿的操作父 live，则所有未关闭 depth d 腿都能通过 persistent AncOK。

证明

对 d 归纳。

基础步： $d = 0$

无操作父，AncOK 条件为空：

$$\text{OpAnc}(L) = \emptyset.$$

只要 L live，就通过。

归纳步

假设所有 depth $< d$ 的未关闭腿，只要 live 就能通过。

取 depth d 的腿 L 。它的操作父 $P = \text{opParent}(L)$ depth 为 $d - 1$ 。

由假设：

$$P \in \mathcal{P}_{i+1}^{\text{live}}.$$

由归纳假设， P 通过 AncOK。

又 L 自己 live，且其祖先链由 P 及 P 的祖先构成，因此全部通过。

所以 L 通过。

证毕。

这说明修复后 depth > 0 腿不会因为 full snapshot 重建而系统性归零。

12. 持久层下的全定义策略

现在给出完整策略定义。

状态

定义完整状态：

$$x_i = (D_i, E_i, \mathcal{P}_i, H_i, A_i, p_i, O_i, M_i, \nu_i).$$

其中：

- D_i ：塔结构；
- E_i ：当前 snapshot 元素树；
- $\mathcal{P}_i$ ：persistent registry；
- H_i ：held legs；
- A_i ：活动操作元素/腿集合；
- p_i ：分账本头寸；
- O_i ：订单；
- M_i ：记忆；
- ν_i ：执行、保证金、借券等状态。

更新流程

第一步，结构更新：

$$D_{i+1} = \text{BU}_{\Theta}(D_i, Y_{i+1}).$$

第二步，生成当前快照：

$$E_{i+1} = \text{Extract}_{\Theta}(D_{i+1}).$$

第三步，持久层对账：

$$\mathcal{P}_{i+1} = \text{Reconcile}_{\Theta}(\mathcal{P}_i, E_{i+1}, H_i, M_i).$$

第四步，当前区间套证书：

$$\Gamma_{i+1} = \text{TD}_{\Theta}(D_{i+1}, \mathcal{P}_{i+1}, A_i).$$

第五步，解释器输出开关集合：

$$(\mathcal{D}_{i+1}, \mathcal{B}_{i+1}, \mathcal{L}_{i+1}) = \mathcal{I}_{\Theta}(\Gamma_{i+1}, A_i, \mathcal{P}_{i+1}).$$

第六步，活动集合更新：

$$A_{i+1} = \text{AncOK}_{\mathcal{P}_{i+1}}[(A_i \setminus \mathcal{D}_{i+1}) \cup \mathcal{B}_{i+1}].$$

第七步，目标分账本头寸：

$$\tilde{p}_{i+1} = \text{LegTarget}_{\Theta}(A_{i+1}, \mathcal{P}_{i+1}).$$

第八步，风险可行集：

$$\mathcal{K}_{\Theta}(x_{i+1}) \subseteq \mathcal{P}^{\text{sep}}.$$

第九步，唯一风险投影：

$$p_{i+1}^* = \text{LexArgmin}_{p \in \mathcal{K}_{\Theta}(x_{i+1})} J_{i+1}(p, \tilde{p}_{i+1}).$$

第十步，订单：

$$O_{i+1} = \text{Schedule}_{\Theta}(p_{i+1}^* - p_i).$$

因此策略为：

$$\pi_{\Theta}(x_{i+1}) = \text{Schedule}_{\Theta} \left[\text{LexArgmin}_{p \in \mathcal{K}_{\Theta}(x_{i+1})} J_{i+1}(p, \tilde{p}_{i+1}) - p_i \right].$$

13. 全定义策略证明

定理 3

若以下条件成立：

1. BU_{Θ} 是函数；
2. Extract_{Θ} 是函数；
3. $\text{Reconcile}_{\Theta}$ 是函数；
4. TD_{Θ} 输出有限集合；
5. $\mathcal{I}_{\Theta}$ 是确定性有限解释器；
6. $\text{AncOK}_{\mathcal{P}}$ 是函数；
7. $\text{LegTarget}_{\Theta}$ 是函数；
8. $\mathcal{K}_{\Theta}(x) \neq \emptyset$ ，且在手数网格上有限；
9. LexArgmin 有固定平局规则；
10. Schedule_{Θ} 是函数；

则：

$$\boxed{\forall x_i, \quad \exists! O_{i+1} = \pi_{\Theta}(x_i).}$$

证明

由 1:

$$D_{i+1}$$

唯一。

由 2:

$$E_{i+1}$$

唯一。

由 3:

$$\mathcal{P}_{i+1}$$

唯一。

由 4:

$$\Gamma_{i+1}$$

唯一且有限。

由 5:

$$(\mathcal{D}_{i+1}, \mathcal{B}_{i+1}, \mathcal{L}_{i+1})$$

唯一。

由 6:

$$A_{i+1}$$

唯一。

由 7:

$$\tilde{p}_{i+1}$$

唯一。

由 8, $\mathcal{K}_{\Theta}(x_{i+1})$ 非空有限, 所以 J 在其上存在最小值。

由 9, 若最小值多个, 字典序选出唯一一个, 因此:

$$p_{i+1}^*$$

唯一。

由 10:

$$O_{i+1}$$

唯一。

证毕。

14. 完全互斥分类证明

定义分类函数:

$$\mathfrak{C}_\Theta(x_i) = (D_i, E_i, \mathcal{P}_i, \Omega_i, \Gamma_i, \mathcal{I}_\Theta(\Gamma_i, A_i, \mathcal{P}_i), A_i, \tilde{p}_i).$$

其中 Ω_i 是所有 held references 的四态分类：

$$\Omega_i(u) \in \{\text{LivePresent}, \text{LiveDetached}, \text{Closed}, \text{Invalidated}\}.$$

因为每个组件都是函数，所以：

$$\mathfrak{C}_\Theta$$

是函数。

定义类别：

$$C_c = \mathfrak{C}_\Theta^{-1}(c).$$

则：

$$\mathcal{X}_\Theta = \bigsqcup_{c \in \text{im } \mathfrak{C}_\Theta} C_c.$$

证明：

任意 $x \in \mathcal{X}_\Theta$ ，因为 $\mathfrak{C}_\Theta$ 是函数，所以存在唯一：

$$c = \mathfrak{C}_\Theta(x).$$

因此：

$$x \in C_c.$$

若：

$$c \neq c',$$

则不可能同时有：

$$x \in C_c \quad \text{和} \quad x \in C_{c'}.$$

否则：

$$\mathfrak{C}_\Theta(x) = c$$

且：

$$\mathfrak{C}_\Theta(x) = c',$$

与函数单值性矛盾。

所以原像族互斥且穷尽。

证毕。

15. 为什么这个修复保持“不伪造根”

持久层不是把 missing parent 伪造成 None。

它做的是区分：

当前 snapshot 中找不到

和：

持久操作对象已经失效

原 Q4 只有两类：

present vs stale.

修复后至少有四类：

LivePresent, LiveDetached, Closed, Invalidated.

其中：

LiveDetached

不是 root，也不是 fake parent=None。

它的 parent 仍然是：

$\text{opParent}(L)$.

只是当前 snapshot 没有展示这个元素。

所以它同时满足：

不伪造根

和：

不因 snapshot 重构误杀 held 腿。

16. 最小修复与完整修复

最小修复：persistent overlay

保留：

$$E_i = \text{Extract}(D_i)$$

每 bar 全量重建。

新增：

$$\mathcal{P}_i.$$

更新：

$$\mathcal{P}_{i+1} = \text{Merge}(\mathcal{P}_i, E_{i+1}, H_i).$$

规则：

1. snapshot 命中则刷新 persistent object 的 snapshot 字段；
2. held 引用但 snapshot 不命中，则标记为 **LiveDetached**，不 stale；
3. 只有显式 close / risk close / invalidation 才退出 live；
4. AncOK 使用 persistent operation parent；
5. 不允许 parent=None 伪造。

这能最快验证：

$$\text{depth} > 0 \text{ legs admission} > 0.$$

完整修复：增量 extract

最终更干净的模型是：

$$\mathcal{P}_{i+1} = \text{IncExtract}(\mathcal{P}_i, Y_{i+1}).$$

已确认元素持久：

$$e \in \mathcal{P}_i^{\text{confirmed}} \implies e \in \mathcal{P}_{i+1}^{\text{confirmed}}$$

且：

$$\text{pid}_{i+1}(e) = \text{pid}_i(e).$$

更高级别涌现时：

$$g_1, \dots, g_m \mapsto G$$

只新增：

$$G$$

和包含边：

$$g_j \sqsubset G,$$

不删除：

$$g_j.$$

这对应严格语法：

高级别涌现新增包含关系，不改写低级别元素。

17. 最终结论

严格推导得到：

Q4 后仍 FAIL 的必要链条是：

per-bar 重建 $E_i \Rightarrow$ 旧 held ID 不在 $E_{i+1} \Rightarrow \text{Stale} \Rightarrow \text{prune} \Rightarrow$ 父链缺失 $\Rightarrow \text{AncOK}$ 剪掉 $\text{depth} >$

这个链条在 snapshot-only 架构下是数学必然，不是参数偶然。

Q4 修复了：

ID 确定性 + 不伪造 root

但缺少：

跨 bar live identity persistence.

严格修复必须引入：

$\mathcal{P}_i = \text{persistent element registry.}$

并把 held-leg validity 从：

$$\epsilon(L) \in \text{Id}(E_i)$$

改成：

$$\epsilon(L) \in \mathcal{P}_i \wedge \neg \text{Invalidated}_i(L).$$

同时把 missing snapshot 分成：

LiveDetached

而不是直接判：

Stale.

在这些不变量下，可以严格证明：

未显式关闭的 $\text{depth} > 0$ 腿不会因全量重建 snapshot 而被系统性剪掉。

并且整个策略仍满足：

$$\forall x_i, \quad \exists! O_{i+1} = \pi_{\Theta}(x_i),$$

以及：

$$\mathcal{X}_{\Theta} = \bigsqcup_c \mathfrak{C}_{\Theta}^{-1}(c).$$

也就是：全定义、全互斥、且不伪造根的严格操作语义。

